Mémo de mise à jour serveur de production — 25/08/2026

1. Résumé des changements du jour

Analyse basée sur les dates de modification des fichiers et le contenu du commit initial (`f234f28`). Le changement majeur de la journée est la **migration du moteur de base de données MySQL → PostgreSQL**, accompagnée de plusieurs évolutions fonctionnelles.

Domaine	Changement
DB — moteur	Passage de MySQL à PostgreSQL. Nouveau schéma versionné : `backend/sql/postgres/schema.sql` . `backend/config/db.php` utilise désormais un DSN PDO `pgsql:` . Les anciens fichiers `base.sql` / `parc_info.sql` (MySQL) sont obsolètes, à ne plus utiliser comme référence.
DB — historique d'affectation matériel	Nouvelle table `t_materiel_affectations` (historique utilisateur/service dans le temps) — `backend/sql/postgres/add_materiel_affectations.sql` .
DB — dernière affectation	Nouvelle colonne `t_materiel.date_derniere_affect` (DATE), avec back-fill depuis `date_achat` pour les lignes existantes — `backend/sql/postgres/add_date_derniere_affect.sql` .
DB — motif/cause tâche	Nouvelle colonne `t_taches.motif_cause` (TEXT) — `backend/sql/add_motif_cause_column.sql` . ⚠️ **Voir point d'attention ci-dessous : ce script est en syntaxe MySQL et n'est PAS exécutable tel quel sur PostgreSQL.**
Backend — API	`auth.php` , `dashboard.php` , `materiel.php` , `taches.php` , `utilisateurs.php` , `services.php` , `type_materiel.php` : tous modifiés aujourd'hui (le plus impacté : `materiel.php` , dernière modif 15h56 — probablement lié à l'historique d'affectations).
Backend — sécurité/config	`backend/middleware/auth.php` revu (14h02). `backend/config/env.php` + `.env.example` alignés sur PostgreSQL (`DB_DRIVER=pgsql` , port 5432).
Frontend	`frontend/assets/js/script.js` (111 Ko, dernière modif 15h59 — le plus gros changement du jour), `style.css` , `login.html` , `frontend/index.html` .
Racine du webroot	`.htaccess` ajouté : bloque l'accès HTTP direct aux fichiers `.env*` et `*.sql` . `index.html` / `login.html` à la racine : redirection simple vers `frontend/` .
Documentation	5 README ajoutés sous `docs/` .
Scripts d'admin	`backend/scripts/create_sessions_table.php` , `create_taches_table.php` , `create_utilisateurs_table.php` , `update_taches_table.php` , `update_passwords.php` — scripts ponctuels utilisés pendant la migration (pas destinés à tourner automatiquement en prod, cf. point d'attention).

Le dépôt Git n'existait pas avant aujourd'hui : le commit `f234f28` est un **commit initial** regroupant tout ce qui précède. Il n'y a donc pas d'historique antérieur permettant un diff plus fin — l'analyse ci-dessus repose sur les dates de modification (mtime) des fichiers.

2. Points d'attention avant mise en prod

1. **`add_motif_cause_column.sql` est incompatible PostgreSQL.** Contenu actuel : `ALTER TABLE t_taches ADD COLUMN motif_cause TEXT AFTER titre;` — la clause `AFTER` n'existe pas en PostgreSQL (elle vient de la syntaxe MySQL). Si la base de prod ne l'a pas déjà, exécuter à la place : `ALTER TABLE t_taches ADD COLUMN motif_cause TEXT;` → Non bloquant si la prod est réinitialisée à partir de `schema.sql` (qui contient déjà la colonne), mais **bloquant si c'est une migration incrémentale** sur une base existante.

2. **Le vrai `.env` de production ne doit jamais être dans `htdocs/`.** `db.php` cherche d'abord `/Applications/MAMP/parc-secrets/.env` (hors webroot), et ne retombe sur `<projet>/.env` qu'en secours. En prod, répliquer ce principe : placer le `.env` réel **hors du répertoire servi par le serveur web**, avec les vrais identifiants PostgreSQL de prod (pas ceux de `.env.example`, qui sont des valeurs de dev).

3. **Scripts `backend/scripts/*.php` = outils ponctuels, pas des endpoints à exposer.** Ils créent/modifient des tables ou rehachent des mots de passe. Ne pas les rendre accessibles publiquement une fois la migration faite (ou les supprimer/déplacer hors webroot après usage), sinon risque de ré-exécution accidentelle ou d'accès non authentifié.

4. **Sessions.** L'auth combine `session_start()` (session PHP native) **et** une table `t_sessions` avec token custom (expiration 24h). Vérifier en prod : - que le répertoire de sessions PHP est accessible en écriture par le process web, - que le nettoyage des sessions expirées (`DELETE ... WHERE expiry_time < NOW()`) continue de s'exécuter (il est fait à chaque login, pas de cron dédié).

5. `.htaccess` ne protège que sous Apache avec `AllowOverride` activé pour ce répertoire — à vérifier sur le serveur de prod (souvent différent de MAMP local).

3. Étapes de mise à jour du serveur de production

3.1 Avant la bascule

- □ **Sauvegarder la base de production actuelle** (dump complet), quel que soit le moteur actuellement en place.
- □ Confirmer le moteur DB réellement utilisé en prod aujourd'hui (MySQL historique ou déjà PostgreSQL). Si c'est encore MySQL, cette mise à jour implique une **migration de moteur**, pas une simple mise à jour de schéma — planifier une fenêtre de maintenance, pas un déploiement à chaud.
- □ Préparer/valider le `.env` de prod (driver, host, port, nom de base, user, password) **hors webroot**, sur le modèle de `.env.example`.
- □ Vérifier que l'extension PHP `pdo_pgsql` est installée et activée sur le serveur de prod.

3.2 Base de données

Sur une base PostgreSQL neuve (recommandé si la prod était en MySQL) :

```
psql -h <host> -U <user> -d <dbname> -f backend/sql/postgres/schema.sql
psql -h <host> -U <user> -d <dbname> -f
backend/sql/postgres/add_materiel_affectations.sql
psql -h <host> -U <user> -d <dbname> -f
backend/sql/postgres/add_date_derniere_affect.sql
```

(`schema.sql` contient déjà `motif_cause` — pas besoin de rejouer `add_motif_cause_column.sql` dans ce cas.)

Sur une base PostgreSQL **déjà existante** (migration incrémentale uniquement, sans repartir de `schema.sql`) :

```
psql -h <host> -U <user> -d <dbname> -f
backend/sql/postgres/add_materiel_affectations.sql
psql -h <host> -U <user> -d <dbname> -f
backend/sql/postgres/add_date_derniere_affect.sql
psql -h <host> -U <user> -d <dbname> -c "ALTER TABLE t_taches ADD COLUMN motif_cause
TEXT;"
```

- □ Si migration MySQL → PostgreSQL avec **données existantes à conserver** : prévoir un export/transformation des données (les scripts SQL fournis créent le schéma vide, pas de script de migration de données automatisé n'a été trouvé dans le repo).
- □ Créer/vérifier le compte utilisateur admin initial si base neuve (`backend/scripts/create_utilisateurs_table.php` ou insertion manuelle avec mot de passe haché via `password_hash`).

3.3 Code applicatif

- □ `git pull` (ou déploiement du commit `f234f28` / dernier commit) sur le serveur de prod.
- □ Déposer le `.env` de prod hors du webroot (chemin équivalent à `/Applications/MAMP/parc-secrets/.env` côté dev — adapter le chemin en dur dans `backend/config/db.php` si l'arborescence de prod diffère).
- □ Vérifier les droits fichiers : les scripts `.php` sous `backend/api/` doivent être exécutables par le serveur web ; `.env` en lecture seule pour l'utilisateur du process PHP uniquement.
- □ Vérifier que `AllowOverride All` (ou équivalent) est actif pour que `.htaccess` s'applique réellement (protection de `.env*` et `*.sql`).

3.4 Vérifications post-déploiement

- □ Test de connexion DB : ouvrir une page qui touche `db.php` et vérifier l'absence d'erreur 500 (`Erreur de connexion à la base de données`).
- □ Test de login (`auth.php?action=login`) → vérifier la création de ligne dans `t_sessions` .
- □ Test CRUD matériel avec affectation utilisateur/service → vérifier l'écriture dans `t_materiel_affectations` et la mise à jour de `date_derniere_affect` .
- □ Test création/édition de tâche avec le champ **Motif/Cause** → vérifier l'écriture en base (valide la présence de la colonne `motif_cause`).
- □ Vérifier que `https://<domaine>/.env` et `https://<domaine>/backend/sql/...sql` renvoient bien une erreur d'accès (403), pas le contenu du fichier.

- □ Vérifier que les scripts `backend/scripts/*.php` ne sont pas accessibles publiquement une fois la migration terminée.

3.5 Rollback

- □ En cas d'échec, restaurer le dump DB pré-migration (3.1) et redéployer le commit précédent (si applicable) ou couper le trafic vers la nouvelle version le temps de corriger.

Mémo généré à partir de l'analyse des fichiers modifiés le 25/08/2026 et du commit `f234f28`. À faire valider par la personne ayant réalisé la migration MySQL → PostgreSQL avant exécution en production, notamment sur le point 2.2 (données existantes) et le chemin réel du `.env` de prod.